

Learning Representations for Graph Retrieval

Omm Arindam

24110229@iitgn.ac.in

Indian Institute of Technology
Gandhinagar

Gandhinagar, Gujarat, India

Priyanshu Raj

priyanshu.raj@iitgn.ac.in

Indian Institute of Technology
Gandhinagar

Gandhinagar, Gujarat, India

Ramji Purwar

ramji.purwar@iitgn.ac.in

Indian Institute of Technology
Gandhinagar

Gandhinagar, Gujarat, India

Abstract

In this project, we study graph retrieval, where the goal is to efficiently find graphs in a large corpus that are most similar to a query graph. Since exact similarity measures are computationally expensive, we learn graph representations using a Graph Isomorphism Network (GIN) and perform approximate nearest neighbor search using Locality Sensitive Hashing (LSH). We evaluate our approach on multiple benchmark datasets and analyze retrieval quality and efficiency across different settings.

Keywords

graph retrieval, graph embeddings, locality sensitive hashing, graph neural networks, approximate nearest neighbor search

1 Introduction

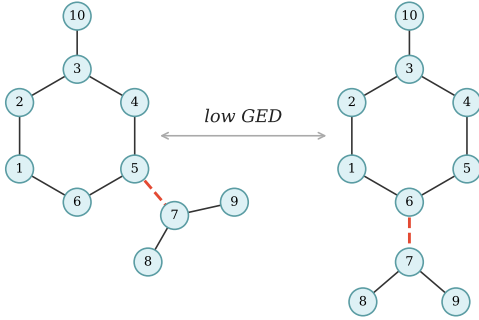


Figure 1: Two graphs with low graph edit distance (GED). Small structural changes result in similar graph structures.

Graphs are a powerful representation for relational data, including social networks, molecular structures, communication networks, and recommendation systems. As graph datasets grow in size, it becomes increasingly difficult to efficiently store, analyze,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CS 328, Gandhinagar, India

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X

<https://doi.org/10.1145/XXXXXXX.XXXXXXX>

and compare such large collections. This makes graph retrieval particularly challenging at scale.

Traditional approaches rely on exact similarity measures such as Graph Edit Distance (GED), which capture structural differences between graphs but are computationally expensive and do not scale well to large corpora.

As illustrated in Figure 1, even small changes in edges or node connections can lead to slightly different graphs that remain structurally similar. This motivates the need for methods that can efficiently approximate graph similarity while preserving important structural properties.

This motivates the use of learned graph representations and approximate nearest neighbor search techniques. By encoding graphs into dense vector embeddings, similarity computation can be reduced to efficient distance calculations in a continuous space.

In this work, we combine Graph Neural Networks (GNNs) for learning graph embeddings with Locality Sensitive Hashing (LSH) for scalable retrieval, and evaluate the approach across multiple benchmark datasets.

2 Problem Statement

We consider the problem of graph retrieval over a large corpus of graphs.

Given a query graph Q and a corpus $\mathcal{G} = \{G_1, G_2, \dots, G_n\}$, we aim to retrieve the top- k graphs that are most similar to Q .

Graph similarity is commonly defined using measures such as Graph Edit Distance (GED), which quantifies the minimum number of edit operations required to transform one graph into another.

However, computing GED exactly is computationally expensive and does not scale to large datasets.

Objective. For a query graph Q , we seek to learn a ranking function $r(Q, G)$ such that graphs with smaller structural distance to Q are ranked higher.

Challenge. The key challenge is to balance retrieval accuracy with computational efficiency. Exact methods provide high-quality results but are prohibitively slow, while approximate methods must preserve structural similarity without significant loss in performance.

Approach. To address this, we learn graph representations that enable efficient similarity computation and scalable retrieval over large graph corpora.

3 Methodology

3.1 Overview

Our approach consists of two main stages: (i) learning graph embeddings using a Graph Neural Network (GNN), and (ii) performing efficient similarity search using Locality Sensitive Hashing (LSH). The overall pipeline converts graphs into vector representations and retrieves similar graphs using a combination of approximate and exact search.

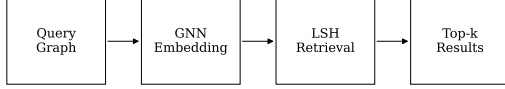


Figure 2: Graph retrieval pipeline: query graph $\rightarrow$ GNN embedding $\rightarrow$ LSH-based candidate retrieval $\rightarrow$ re-ranking $\rightarrow$ top-k results.

3.2 Graph Embedding using GIN

We use a Graph Isomorphism Network (GIN) to encode graphs into fixed-dimensional vectors. Given a graph $G = (V, E)$, each node $v \in V$ is initialized with a feature vector $\mathbf{h}_v^{(0)}$.

The GIN updates node representations over multiple layers using message passing. At each layer k , a node aggregates information from its neighbors and updates its representation:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)}) \cdot \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)} \right)$$

Here, $\mathcal{N}(v)$ denotes the neighbors of node v , and $\epsilon^{(k)}$ controls the weighting of the node's own features.

In our implementation, we use a 3-layer GIN:

- Layer 1 aggregates immediate neighbors (1-hop structure)
- Layer 2 captures 2-hop neighborhood information
- Layer 3 captures higher-order structural patterns

After the final layer, node embeddings are aggregated into a graph-level embedding using sum pooling:

$$\mathbf{z}_G = \sum_{v \in V} \mathbf{h}_v^{(K)}$$

This produces a fixed-dimensional vector representation for each graph.

3.3 Triplet Construction and GED Oracle

Triplet construction requires a similarity oracle to determine whether two graphs are structurally similar or dissimilar. We use Graph Edit Distance (GED) as the underlying similarity measure, approximated using beam search.

Computing exact GED is NP-hard and infeasible for large datasets. Therefore, we use the beam search algorithm of Neuhaus et al. to

obtain approximate GED values in polynomial time. Beam search explores possible edit paths while keeping only the top- B candidates at each step, significantly reducing computation while maintaining reasonable accuracy.

Training Oracle. During training, we use approximate GED with a small beam width ($B = 5$) to rank graph pairs. This is sufficient to distinguish between relatively similar and dissimilar graphs while keeping computation efficient.

Triples (G_a, G_p, G_n) are constructed as follows:

- Positive graph (G_p): has low GED with respect to anchor G_a
- Negative graph (G_n): has significantly higher GED

Due to approximation errors in GED, the labels may be noisy. To account for this, we use a conservative margin $\alpha = 1.0$ in the triplet loss, allowing some tolerance in the embedding space.

Evaluation Ground Truth. For evaluation, we use a more accurate oracle:

- Exact GED (via GEDLib) for small graphs (≤ 30 nodes)
- Beam search GED with larger beam width ($B = 20$) for larger graphs

Oracle Separation. It is important to separate the training oracle from the evaluation ground truth. Training and evaluating using the same approximate GED would only measure consistency with the approximation rather than true structural similarity.

By using more accurate GED estimates during evaluation, we obtain a more reliable measure of retrieval performance and a better approximation of true graph similarity.

3.4 Triplet Loss Optimization

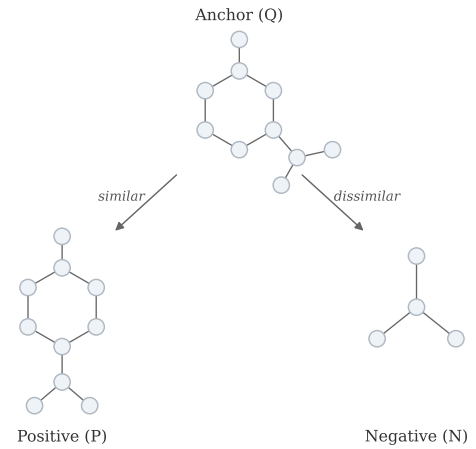


Figure 3: Triplet training setup. The anchor graph is pulled closer to the positive graph and pushed away from the negative graph in embedding space.

Let $\mathbf{z}_a$, $\mathbf{z}_p$, and $\mathbf{z}_n$ denote embeddings of anchor, positive, and negative graphs.

We use the triplet loss function:

$$\mathcal{L} = \max(0, d(\mathbf{z}_a, \mathbf{z}_p) - d(\mathbf{z}_a, \mathbf{z}_n) + \alpha)$$

where $d(\cdot, \cdot)$ is Euclidean distance and α is a margin.

This loss enforces that:

$$d(\mathbf{z}_a, \mathbf{z}_p) + \alpha < d(\mathbf{z}_a, \mathbf{z}_n)$$

Thus, similar graphs are pulled closer in the embedding space, while dissimilar graphs are pushed apart. Over training, this organizes the embedding space such that structurally similar graphs form clusters.

3.5 LSH-based Approximate Retrieval

Given graph embeddings $\mathbf{z} \in \mathbb{R}^d$, we use LSH to enable efficient retrieval.

Each hash function projects the embedding onto a random direction:

$$h_{a,b}(\mathbf{z}) = \left\lfloor \frac{\mathbf{a}^\top \mathbf{z} + b}{w} \right\rfloor$$

where $\mathbf{a}$ is a random vector and w is the bucket width.

To increase selectivity, we concatenate k such hash functions:

$$g(\mathbf{z}) = (h_1(\mathbf{z}), \dots, h_k(\mathbf{z}))$$

To improve recall, we use L independent hash tables.

Parameter Roles.

- k : controls selectivity (larger $k \rightarrow$ smaller buckets)
- L : controls recall (larger $L \rightarrow$ more candidates)

3.6 Candidate Generation and Retrieval Pipeline

Given a query graph Q , we first compute its embedding $\mathbf{z}_Q$.

Candidate Generation. The embedding is hashed into each of the L hash tables. For each table, we retrieve all graphs in the same bucket. The union across all tables forms the candidate set:

$$C = \bigcup_{i=1}^L \text{Bucket}_i(\mathbf{z}_Q)$$

This reduces the search space significantly, i.e., $|C| \ll |\mathcal{G}|$.

Re-ranking. We compute exact distances between the query and each candidate:

$$d(\mathbf{z}_Q, \mathbf{z}_G), \quad \forall G \in C$$

Candidates are sorted based on this distance, and the top- k nearest graphs are returned.

4 Datasets

We evaluate our approach on the following standard graph classification datasets:

- **MUTAG**: A small molecular dataset where graphs represent chemical compounds. Nodes correspond to atoms and edges represent chemical bonds. The task is to predict mutagenicity.
- **PROTEINS**: A bioinformatics dataset where graphs represent protein structures. Nodes correspond to secondary structure elements, and edges represent spatial proximity.
- **AIDS**: A molecular dataset containing chemical compounds tested for activity against HIV. Graphs encode molecular structures with labeled nodes and edges.
- **IMDB-Binary**: A social network dataset derived from movie collaboration graphs. Nodes represent actors, and edges indicate co-appearances in movies. Graphs are unlabeled and structurally sparse.
- **Reddit-Binary**: A large-scale social network dataset where graphs represent discussion threads. Nodes correspond to users and edges represent interactions.

Table 1: Dataset statistics

Dataset	Graphs	Avg Nodes	Avg Edges
MUTAG	188	17.9	19.8
PROTEINS	1113	39.1	72.8
AIDS	2000	15.7	16.2
IMDB-Binary	1000	19.8	96.5
Reddit-Binary	2000	429.6	497.8

5 Evaluation Metrics

We evaluate the performance of the graph retrieval system using standard ranking and efficiency metrics. These metrics capture both the quality of retrieved results and the efficiency of the retrieval process.

5.1 Precision@k

Precision@ k measures the fraction of relevant graphs among the top- k retrieved results:

$$\text{Precision@}k = \frac{|R_k \cap R^*|}{k}$$

where R_k is the set of top- k retrieved graphs and R^* is the set of relevant graphs.

Precision@ k evaluates how accurate the top results are. A high value indicates that most of the retrieved graphs are structurally similar to the query graph.

5.2 Recall@k

Recall@ k measures the fraction of all relevant graphs that are successfully retrieved:

$$\text{Recall}@k = \frac{|R_k \cap R^*|}{|R^*|}$$

$\text{Recall}@k$ evaluates coverage. It indicates how many of the truly similar graphs are captured within the top- k results.

5.3 Average Precision (AP)

Average Precision (AP) measures ranking quality for a single query by considering the order of retrieved results:

$$\text{AP} = \frac{1}{|R^*|} \sum_{i=1}^k \text{Precision}@i \cdot \text{rel}(i)$$

where $\text{rel}(i) = 1$ if the item at rank i is relevant, and 0 otherwise.

AP rewards methods that rank relevant graphs higher. Early correct retrievals contribute more to the score.

5.4 Mean Average Precision (MAP)

Mean Average Precision (MAP) aggregates AP over all queries:

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

where Q is the set of query graphs.

MAP provides a global measure of ranking quality across the entire dataset.

5.5 Approximation Quality (AQ)

To evaluate the effectiveness of LSH compared to exact brute-force (BF) retrieval, we define Approximation Quality as:

$$\text{AQ} = \frac{|R_k^{\text{LSH}} \cap R_k^{\text{BF}}|}{k}$$

where R_k^{LSH} and R_k^{BF} denote the top- k results from LSH and brute-force search, respectively.

AQ measures how closely the approximate retrieval matches the exact results. A value close to 1 indicates that LSH retrieves nearly the same neighbors as brute-force search.

5.6 Query Time

Query time measures the average time required to process a query and retrieve the top- k results.

This metric reflects the efficiency of the retrieval system. While brute-force search guarantees exact results, it is computationally expensive. LSH significantly reduces query time by limiting the search to a smaller candidate set.

5.7 Baselines for Comparison

To evaluate the effectiveness of the proposed approach, we compare against the following baselines:

- **Brute-Force (BF):** Exact nearest neighbor search performed by computing pairwise distances between the query embedding and all graphs in the corpus. This serves as the upper bound for retrieval accuracy.
- **LSH-based Retrieval:** Approximate nearest neighbor search using Locality Sensitive Hashing on learned embeddings. This is the primary method evaluated for efficiency and approximation quality.
- **Untrained GNN:** Graph embeddings obtained from the same GNN architecture without training. This baseline evaluates the impact of learning on retrieval performance.
- **Graph2Vec:** A widely used unsupervised graph embedding method. This serves as a strong baseline for comparing learned representations.

All methods use the same distance metric in the embedding space to ensure a fair comparison.

6 RESULTS

We evaluate the proposed graph retrieval pipeline across multiple datasets using the metrics defined earlier. The analysis focuses on four key aspects: retrieval accuracy, approximation behavior of LSH, efficiency, and embedding quality.

6.1 Retrieval Accuracy

Table 2: Precision@k across datasets.

k = 5				
Dataset	BF	LSH	Untrained	G2V
MUTAG	0.971	0.968	0.919	0.894
PROTEINS	0.805	0.803	0.664	0.662
AIDS	0.855	0.860	0.839	0.850
IMDB-Binary	0.887	0.841	0.885	0.816
Reddit-Binary	0.843	0.845	0.758	0.604
k = 10				
Dataset	BF	LSH	Untrained	G2V
MUTAG	0.965	0.968	0.869	0.819
PROTEINS	0.797	0.790	0.647	0.654
AIDS	0.837	0.838	0.799	0.815
IMDB-Binary	0.886	0.829	0.870	0.769
Reddit-Binary	0.840	0.840	0.741	0.595
k = 20				
Dataset	BF	LSH	Untrained	G2V
MUTAG	0.950	0.966	0.778	0.733
PROTEINS	0.783	0.777	0.633	0.643
AIDS	0.817	0.816	0.759	0.772
IMDB-Binary	0.872	0.822	0.840	0.698
Reddit-Binary	0.840	0.840	0.726	0.579

Precision@10 evaluates the fraction of relevant graphs retrieved within the top 10 results. Table 2 and Figure 4 show that the trained model consistently achieves high precision across datasets. This

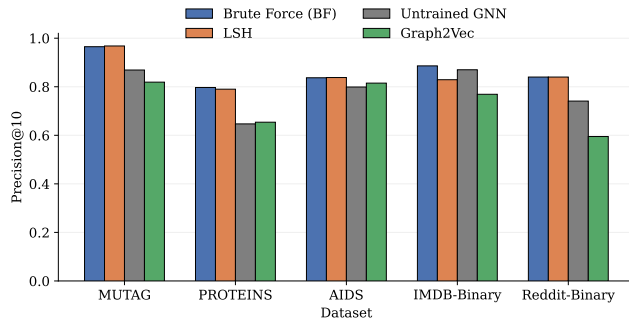


Figure 4: Precision@10 comparison across methods.

indicates that the learned embeddings effectively capture graph similarity, allowing the system to retrieve structurally similar graphs in the top-ranked positions.

Comparing trained embeddings with the untrained baseline reveals a clear improvement across all datasets. For example, on MUTAG, precision increases from 0.869 to 0.968, and on PROTEINS from 0.647 to 0.790. This demonstrates that the training objective successfully shapes the embedding space to reflect meaningful structural relationships between graphs.

The comparison between brute-force (BF) and LSH shows that LSH closely matches BF performance in most cases. Since BF performs exact nearest neighbor search in the embedding space, this result indicates that LSH is able to approximate nearest neighbors effectively without significant loss in accuracy. The only notable degradation is observed for IMDB-Binary, where LSH precision drops from 0.886 (BF) to 0.829. This deviation motivates a deeper analysis of approximation behavior.

6.2 Approximation Behavior

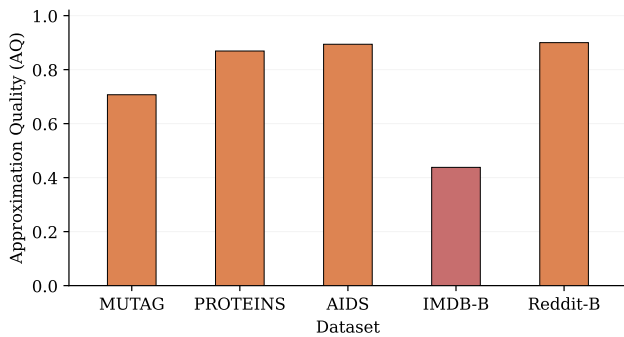


Figure 5: Approximation Quality (AQ) of LSH across datasets.

Approximation Quality (AQ) measures the overlap between the top- k results returned by LSH and brute-force search. A value close to 1 indicates that LSH retrieves nearly the same neighbors as exact search.

As shown in Figure 5, AQ remains high for most datasets, with values above 0.85 for PROTEINS, AIDS, and Reddit-Binary. This

confirms that LSH is able to preserve the nearest neighbor structure in the embedding space for these datasets.

However, IMDB-Binary shows a significantly lower AQ (approximately 0.44), indicating that LSH fails to retrieve many of the true nearest neighbors identified by brute-force search. This suggests that the issue is not with the embedding quality alone, but with how LSH partitions the embedding space for this dataset.

6.3 Candidate Set Analysis

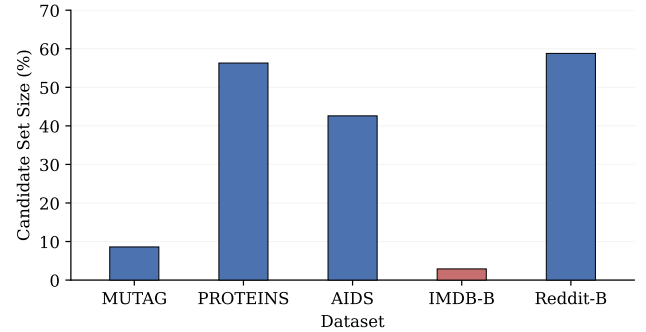


Figure 6: Candidate set size as a percentage of dataset size.

To understand the degradation observed in IMDB-Binary, we analyze the size of the candidate set retrieved by LSH before ranking. The candidate set consists of all graphs that fall into the same hash buckets as the query across multiple hash tables.

Figure 6 shows that for most datasets, the candidate set covers a substantial portion of the dataset, typically between 40% and 60%. This ensures that LSH has access to a sufficiently large pool of potential neighbors, increasing the likelihood of retrieving the true nearest neighbors.

In contrast, IMDB-Binary exhibits extreme candidate starvation, with only about 2.9% of the dataset being considered during retrieval. This drastically reduces the probability of including the true nearest neighbors in the candidate set, leading to lower AQ and reduced precision. This behavior indicates that the effectiveness of LSH is highly dependent on the distribution of embeddings and how well they align with the hash functions.

6.4 Efficiency

Figure 7 compares the query time required to retrieve top- k results using brute-force and LSH. Contrary to the typical expectation that approximate methods are faster, brute-force search is competitive and in some cases faster than LSH in our experiments.

This behavior can be attributed to the relatively small size of the datasets used. Brute-force search involves computing distances between the query and all embeddings, which is efficient when the dataset size is limited. In contrast, LSH introduces additional overhead due to hash computations, bucket lookups, and candidate set aggregation.

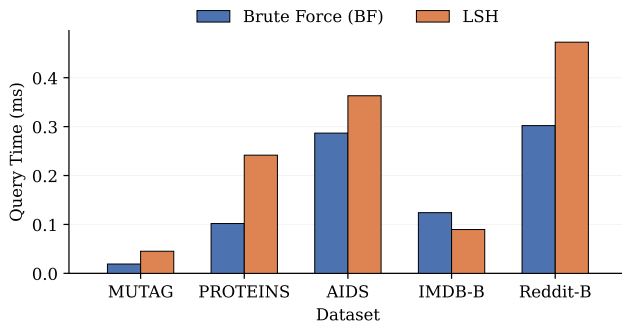


Figure 7: Query time comparison between brute-force and LSH.

As a result, the theoretical advantage of LSH in reducing search complexity is not fully realized at this scale. However, LSH still provides a structured mechanism for limiting the search space, and its benefits are expected to become more pronounced for larger datasets where brute-force search becomes computationally expensive.

6.5 Embedding Quality

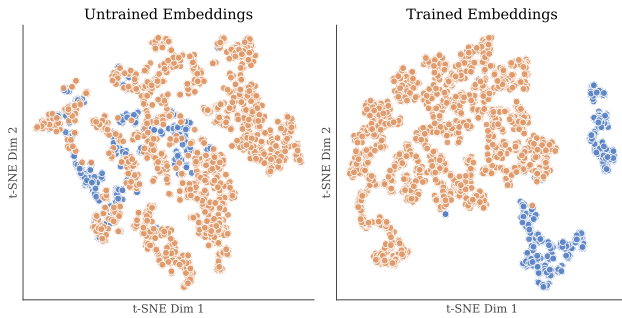


Figure 8: t-SNE visualization for the AIDS dataset.

To qualitatively assess the learned embeddings, we visualize them using t-SNE for the AIDS dataset. In the untrained case, embeddings from different classes are highly intermixed, with no clear separation. Points belonging to different classes occupy overlapping regions of the space, indicating that the raw embeddings do not capture meaningful structural distinctions.

After training, the embedding space becomes significantly more organized. Graphs belonging to the same class form compact clusters, and different classes are more clearly separated. In particular, clusters that were previously embedded within larger groups become distinct and isolated, indicating improved discrimination.

This qualitative improvement aligns with the quantitative results observed earlier. The trained embeddings not only improve retrieval accuracy but also exhibit a structured geometry that supports effective nearest neighbor search.

Acknowledgments

The authors thank Prof. Anirban Dasgupta of the Department of Computer Science and Engineering, IIT Gandhinagar, for proposing this problem and for his guidance throughout the project.

References

- [1] Yunsheng Bai, Hao Ding, Song Bian, Ting Chen, Yizhou Sun, and Wei Wang. 2019. SimGNN: A Neural Network Approach to Fast Graph Similarity Computation. In *Proceedings of the 12th ACM International Conference on Web Search and Data Mining (WSDM)*. 31–39.
- [2] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-Sensitive Hashing Scheme Based on p -Stable Distributions. In *Proceedings of the 20th Annual Symposium on Computational Geometry (SoCG)*. 253–262.
- [3] William L. Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [4] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. 604–613.
- [5] Florian Schroff, Dmitry Kalenichenko, and James Philbin. 2015. FaceNet: A Unified Embedding for Face Recognition and Clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 815–823.
- [6] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.
- [7] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.

A Additional Results

This appendix provides complete quantitative results across all datasets and methods for different values of k . These tables complement the summarized results presented in the main paper.

A.1 Results at $k = 5$

Table 3: Performance metrics at $k = 5$.

Dataset	Method	P@5	R@5	MAP	AQ	QT
MUTAG	BF	0.971	0.066	0.064	N/A	0.019
MUTAG	LSH	0.968	0.042	0.042	0.699	0.047
MUTAG	Untrained	0.919	0.043	0.042	0.800	0.079
MUTAG	Graph2Vec	0.894	0.050	0.048	0.798	0.117
PROTEINS	BF	0.805	0.007	0.007	N/A	0.101
PROTEINS	LSH	0.803	0.006	0.005	0.793	0.224
PROTEINS	Untrained	0.664	0.005	0.004	0.799	N/A
PROTEINS	Graph2Vec	0.662	0.004	0.004	0.746	0.175
AIDS	BF	0.855	0.009	0.008	N/A	0.284
AIDS	LSH	0.860	0.008	0.007	0.797	0.365
AIDS	Untrained	0.839	0.009	0.008	0.800	N/A
AIDS	Graph2Vec	0.850	0.009	0.008	0.792	0.438
IMDB-Binary	BF	0.887	0.022	0.018	N/A	0.129
IMDB-Binary	LSH	0.841	0.014	0.013	0.430	0.092
IMDB-Binary	Untrained	0.885	0.022	0.021	0.715	N/A
IMDB-Binary	Graph2Vec	0.816	0.015	0.015	0.706	0.234
Reddit-Binary	BF	0.843	0.004	0.004	N/A	0.298
Reddit-Binary	LSH	0.845	0.003	0.003	0.800	0.483
Reddit-Binary	Untrained	0.758	0.003	0.003	0.800	N/A
Reddit-Binary	Graph2Vec	0.604	0.002	0.002	0.722	0.447

A.2 Results at $k = 10$

Table 4: Performance metrics at $k = 10$.

Dataset	Method	P@10	R@10	MAP	AQ	QT
MUTAG	BF	0.965	0.127	0.121	N/A	0.019
MUTAG	LSH	0.968	0.081	0.081	0.707	0.045
MUTAG	Untrained	0.869	0.091	0.085	0.900	N/A
MUTAG	Graph2Vec	0.819	0.095	0.087	0.882	0.116
PROTEINS	BF	0.797	0.014	0.013	N/A	0.102
PROTEINS	LSH	0.790	0.013	0.011	0.869	0.242
PROTEINS	Untrained	0.647	0.011	0.008	0.900	N/A
PROTEINS	Graph2Vec	0.654	0.010	0.008	0.805	0.174
AIDS	BF	0.837	0.017	0.014	N/A	0.287
AIDS	LSH	0.838	0.016	0.013	0.894	0.363
AIDS	Untrained	0.799	0.016	0.014	0.900	N/A
AIDS	Graph2Vec	0.815	0.018	0.015	0.877	0.356
IMDB-Binary	BF	0.886	0.041	0.033	N/A	0.124
IMDB-Binary	LSH	0.829	0.026	0.023	0.438	0.090
IMDB-Binary	Untrained	0.870	0.044	0.039	0.819	N/A
IMDB-Binary	Graph2Vec	0.769	0.030	0.028	0.738	0.234
Reddit-Binary	BF	0.840	0.009	0.008	N/A	0.302
Reddit-Binary	LSH	0.840	0.008	0.007	0.900	0.473
Reddit-Binary	Untrained	0.741	0.007	0.006	0.900	N/A
Reddit-Binary	Graph2Vec	0.595	0.005	0.004	0.767	0.448

A.3 Results at $k = 20$

Table 5: Performance metrics at $k = 20$.

Dataset	Method	P@20	R@20	MAP	AQ	QT
MUTAG	BF	0.950	0.237	0.225	N/A	0.018
MUTAG	LSH	0.966	0.132	0.132	0.590	0.046
MUTAG	Untrained	0.778	0.168	0.151	0.950	N/A
MUTAG	Graph2Vec	0.733	0.165	0.144	0.905	0.116
PROTEINS	BF	0.783	0.028	0.025	N/A	0.107
PROTEINS	LSH	0.777	0.025	0.022	0.910	0.256
PROTEINS	Untrained	0.633	0.022	0.016	0.950	N/A
PROTEINS	Graph2Vec	0.643	0.021	0.016	0.820	0.175
AIDS	BF	0.817	0.033	0.026	N/A	0.297
AIDS	LSH	0.816	0.028	0.022	0.934	0.299
AIDS	Untrained	0.759	0.028	0.023	0.950	N/A
AIDS	Graph2Vec	0.772	0.030	0.025	0.897	0.263
IMDB-Binary	BF	0.872	0.079	0.062	N/A	0.133
IMDB-Binary	LSH	0.822	0.044	0.038	0.398	0.090
IMDB-Binary	Untrained	0.840	0.078	0.070	0.897	N/A
IMDB-Binary	Graph2Vec	0.698	0.050	0.046	0.729	0.237
Reddit-Binary	BF	0.840	0.017	0.016	N/A	0.310
Reddit-Binary	LSH	0.840	0.016	0.015	0.950	0.474
Reddit-Binary	Untrained	0.726	0.014	0.012	0.950	N/A
Reddit-Binary	Graph2Vec	0.579	0.011	0.008	0.759	0.449

Note. Untrained query times are omitted as they use the same LSH index configuration as the trained model. Differences in latency are negligible (< 0.01 ms), and this baseline isolates embedding quality rather than indexing performance.